

Xây dựng và tối ưu hóa mô hình máy học dự đoán giá nhà

Trần Hồ Minh Hải, Trương Văn Thiện, Phan Đức Nhân,
Võ Gia Kiệt

Tóm tắt

Báo cáo này trình bày một quy trình học máy toàn diện để dự đoán giá nhà ở tại Ames, Iowa, sử dụng bộ dữ liệu Kaggle Housing Prices. Phương pháp của chúng tôi kết hợp tiền xử lý dữ liệu chặt chẽ, kỹ thuật xây dựng đặc trưng chiến lược và mô hình hợp thành (ensemble) để nâng cao độ chính xác của dự đoán. Quy trình bắt đầu với việc làm sạch dữ liệu tự động, loại bỏ các đặc trưng có quá nhiều giá trị thiếu và các thuộc tính có phương sai gần bằng không. Tiếp theo, chúng tôi triển khai các kỹ thuật xây dựng đặc trưng nâng cao, bao gồm mã hóa biến phân loại, mã hóa dựa trên mục tiêu (target encoding) cho yếu tố khu vực lân cận, và tạo ra các đặc trưng phái sinh như tuổi nhà, tình trạng cải tạo và các chỉ số hiệu quả về chỗ để xe. Mô hình học máy sử dụng một kiến trúc hợp thành trọng số kết hợp ba thuật toán: XGBoost (trọng số 65%), Hồi quy Lasso (trọng số 20%) và ElasticNet (trọng số 15%). Để đánh giá hiệu suất mô hình một cách vững chắc, chúng tôi sử dụng xác thực chéo (cross-validation) và tối ưu hóa theo Sai số căn bậc hai trung bình (RMSE) trên giá nhà đã được chuyển đổi logarithm, phù hợp với tiêu chí đánh giá của cuộc thi. Giải pháp của chúng tôi chứng minh khả năng xử lý hiệu quả những phức tạp của dữ liệu bất động sản trong thực tế, đạt được hiệu suất cạnh tranh thông qua việc lựa chọn đặc trưng cẩn thận, chuẩn hóa và kết hợp mô hình. Toàn bộ quy trình—from tiền xử lý dữ liệu đến dự đoán hợp thành—cung cấp một khuôn khổ có thể tái sản xuất cho các bài toán phân tích thị trường bất động sản.

Từ khóa: Dự đoán Giá Nhà, Học Hợp thành (Ensemble Learning), Xây dựng Đặc trưng, XGBoost, Hồi quy Chuẩn hóa, Phân tích Bất động sản, Quy trình Học máy.

1. Giới thiệu

1.1. Tổng quan vấn đề

Trong lĩnh vực bất động sản, việc dự đoán chính xác giá nhà ở luôn là một bài toán quan trọng và thiết thực, không chỉ đối với các nhà đầu tư, người mua bán nhà mà còn có ý nghĩa lớn đối với các tổ chức tín dụng và cơ quan quản lý. Bài toán dự đoán giá nhà dựa trên các đặc điểm của bất động sản và môi trường xung quanh đặt ra nhiều thách thức kỹ thuật thú vị trong lĩnh vực học máy và khoa học dữ liệu.

Thực tế cho thấy, khi được hỏi về ngôi nhà mơ ước, người mua thường mô tả những yếu tố cảm tính như vị trí, kiến trúc hay tiện nghi, trong đó các yếu tố kỹ thuật như chiều cao trần tầng hầm hay khoảng cách đến đường sắt lại ít được đề cập. Tuy nhiên, bộ dữ liệu Ames Housing đã chứng minh rằng những yếu tố tưởng chừng như kỹ thuật khô khan này lại có ảnh hưởng không nhỏ đến việc định giá và thương lượng giá cả. Bài toán này không chỉ dừng lại ở việc áp dụng các mô hình hồi quy truyền thống mà đòi hỏi sự sáng tạo trong xử lý dữ liệu, xây dựng đặc trưng và kết hợp đa mô hình để đạt được độ chính xác tối ưu.

1.2. Dữ liệu bài toán

Bộ dữ liệu Ames Housing được sử dụng trong nghiên cứu này là một bộ dữ liệu toàn diện về thị trường bất động sản tại thành phố Ames, Iowa, Hoa Kỳ. Đây được xem như một phiên bản mở rộng và hiện đại hóa của bộ dữ liệu Boston Housing kinh điển, cung cấp những thông tin chi tiết và cập nhật hơn về các ngôi nhà dân dụng. Bộ dữ liệu bao gồm thông tin của khoảng 1500 ngôi nhà với 79 thuộc tính mô tả đầy đủ các khía cạnh khác nhau của mỗi bất động sản.

Các thuộc tính trong bộ dữ liệu được chia thành nhiều nhóm khác nhau, phản ánh toàn diện các yếu tố ảnh hưởng đến giá nhà. Nhóm thông tin tổng quan bao gồm các đặc điểm về vị trí và phân vùng như loại hình nhà ở, diện tích lô đất, và chiều dài mặt tiền. Nhóm đặc điểm ngôi nhà tập trung vào chất lượng xây dựng, năm xây dựng, năm cải tạo và tình trạng bảo trì. Các nhóm thông tin khác mô tả chi tiết về tiện nghi như phòng tắm, phòng ngủ, không gian sinh hoạt, cùng với các tiện ích đi kèm như nhà xe, hồ bơi, và hàng rào.

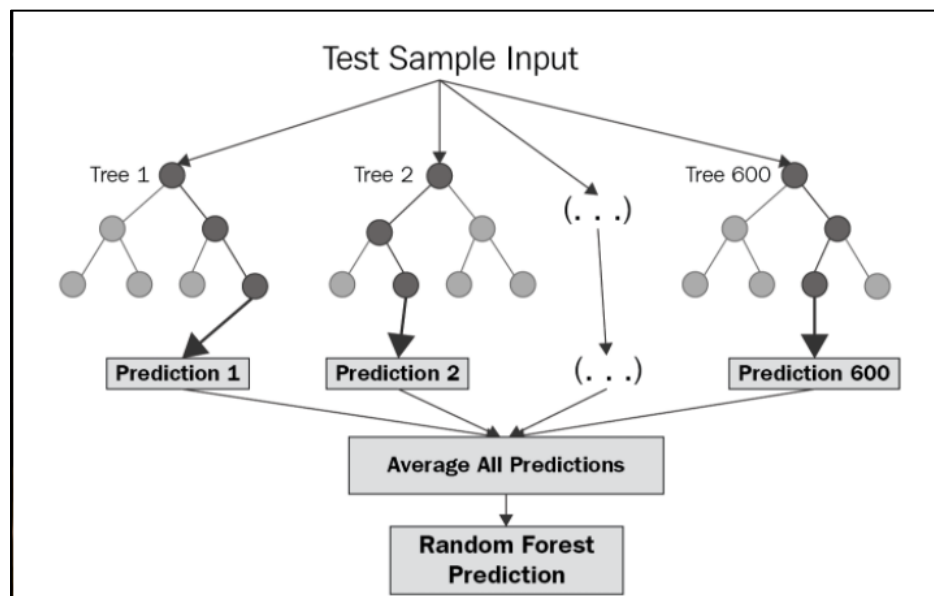
Biến mục tiêu cần dự đoán là giá bán nhà (SalePrice), một biến số có phân phối lệch phải - đặc điểm thường thấy trong các bài toán về giá cả bất động sản. Để xử lý vấn đề này, việc áp dụng phép biến đổi logarithm được thực hiện nhằm chuẩn hóa phân phối và giảm ảnh hưởng của các giá trị ngoại lai. Thách thức chính của bộ dữ liệu này nằm ở việc xử lý các giá trị thiếu, mã hóa biến phân loại, và chuẩn hóa các đặc trưng số, đòi hỏi những kỹ thuật tiền xử lý dữ liệu bài bản và cẩn thận để đảm bảo chất lượng đầu vào cho các mô hình học máy.

2. Công việc liên quan

2.1. Các mô hình máy học

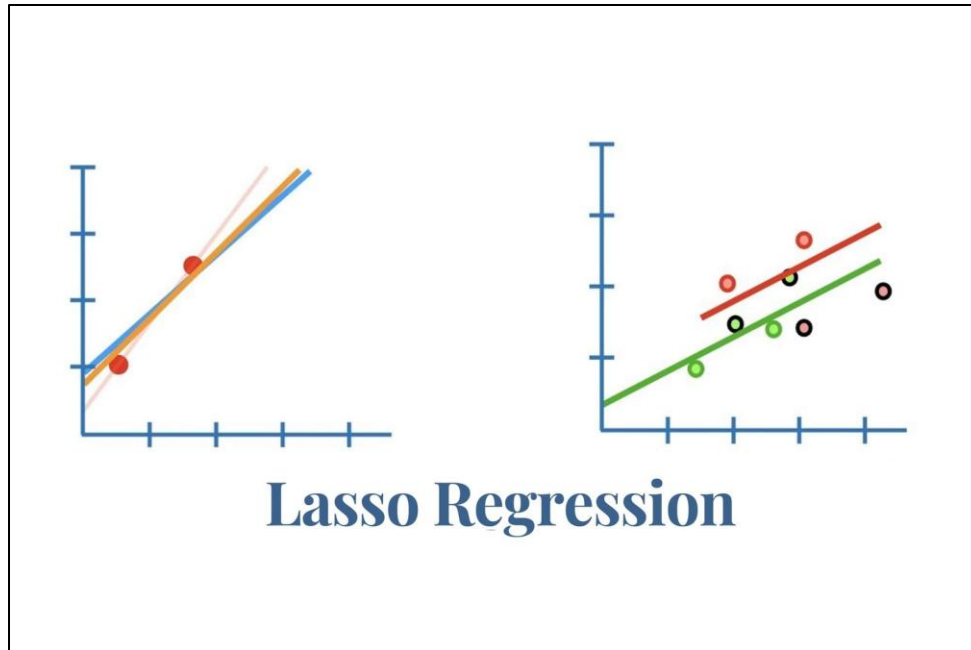
Trong lĩnh vực dự đoán giá nhà, nhiều mô hình máy học đã được nghiên cứu và ứng dụng rộng rãi. Các phương pháp hồi quy tuyến tính truyền thống như Linear Regression thường được sử dụng làm baseline do tính đơn giản và dễ diễn giải. Tuy nhiên, với dữ liệu phức tạp và có nhiều đặc trưng như bộ dữ liệu Ames Housing, các mô hình này thường không đạt được hiệu suất tối ưu do không nắm bắt được các mối quan hệ phi tuyến phức tạp trong dữ liệu.

Để khắc phục những hạn chế này, các mô hình ensemble và cây quyết định đã được phát triển và chứng minh hiệu quả vượt trội. Random Forest Regressor, một trong những mô hình ensemble phổ biến, hoạt động dựa trên nguyên lý kết hợp nhiều cây quyết định để giảm phương sai và cải thiện độ chính xác dự đoán. Mô hình này đặc biệt hiệu quả trong việc xử lý dữ liệu có nhiều đặc trưng và tự động xác định các đặc trưng quan trọng. Tiếp theo, XGBoost (Extreme Gradient Boosting) đại diện cho thế hệ mô hình boosting tiên tiến, với khả năng tối ưu hóa hiệu suất thông qua việc giảm thiểu hàm mất mát một cách tuần tự. XGBoost được biết đến với tốc độ huấn luyện nhanh, khả năng xử lý dữ liệu lớn và hiệu suất dự đoán vượt trội trong nhiều bài toán thực tế.



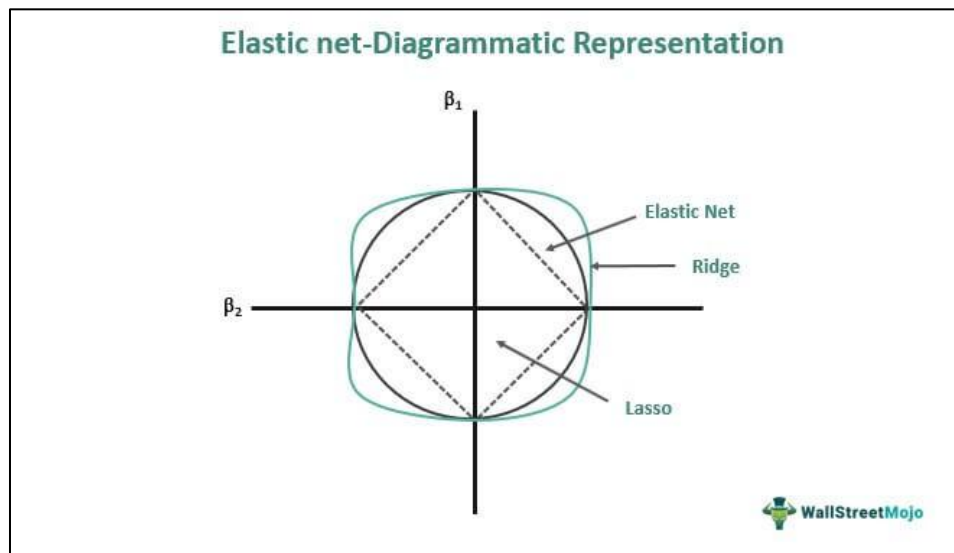
Hình 1. Mô hình Random Forest Regressor

Bên cạnh các mô hình dựa trên cây quyết định, các phương pháp hồi quy có chuẩn hóa cũng đóng vai trò quan trọng. Lasso Regression (L1 regularization) không chỉ thực hiện dự đoán mà còn có khả năng lựa chọn đặc trưng thông qua việc đưa các hệ số không quan trọng về zero. Tính năng này đặc biệt hữu ích khi làm việc với datasets có nhiều đặc trưng, giúp giảm thiểu hiện tượng overfitting.



Hình 2. Mô hình Lasso Regression

Tương tự, ElasticNet kết hợp cả hai chuẩn hóa L1 và L2, mang lại sự cân bằng giữa việc lựa chọn đặc trưng của Lasso và tính ổn định của Ridge Regression. Sự kết hợp này cho phép mô hình xử lý tốt các trường hợp có đa cộng tuyến cao trong dữ liệu.

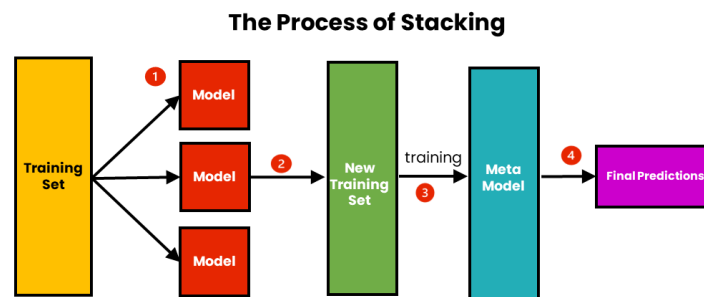


Hình 3. Mô hình ElasticNet

2.2. Các phương pháp Ensemble

2.2.1. Stacking (Stacked Generalization)

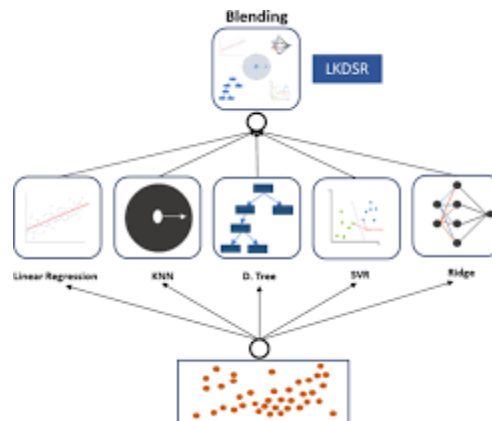
Stacking sử dụng đầu ra của các mô hình cơ sở (base models) làm đặc trưng đầu vào cho một mô hình tổng hợp (meta-model). Meta-model này sẽ học cách kết hợp các dự đoán một cách tối ưu nhất, tạo ra một mô hình mạnh hơn từ các mô hình thành phần.



Hình 5. Phương pháp Stacking

2.2.2. Blending

Blending tương tự Stacking nhưng sử dụng một tập validation (kiểm định) tách biệt, thay vì dùng k-fold cross-validation, để huấn luyện meta-model. Cách tiếp cận này đơn giản hơn, giúp tránh rò rỉ dữ liệu (data leakage) nhưng có thể kém hiệu quả hơn nếu tập dữ liệu nhỏ.



Hình 6. Phương pháp Blending

2.3. Độ đo đánh giá

Việc lựa chọn độ đo đánh giá phù hợp đóng vai trò quan trọng trong việc xác định hiệu suất của các mô hình dự đoán giá nhà. Root Mean Squared Error (RMSE) là độ đo được sử dụng phổ biến nhất trong các bài toán hồi quy, đặc biệt trong lĩnh vực dự đoán giá bất động sản. RMSE tính toán căn bậc hai của trung bình bình phương sai số, cho phép đánh giá mức độ chênh lệch giữa giá trị dự đoán và giá trị thực tế. Độ đo này có ưu điểm là đơn vị tính tương đương với biến mục tiêu,

giúp việc diễn giải kết quả trở nên trực quan và dễ hiểu. Công thức toán học của RMSE được tính như sau:

$$RSME = \sqrt{\frac{\sum (y_i - \hat{y}_i)^2}{N}}$$

Trong đó:

y_i : là giá trị thực tế của mẫu thứ i

$\hat{y}_i$: là giá trị dự đoán của mô hình cho mẫu thứ i

n : là tổng số lượng mẫu trong tập dữ liệu

Σ : biểu thị phép tổng từ $i = 1$ đến n

Tuy nhiên, trong bối cảnh dự đoán giá nhà, nơi mà phân phối của biến mục tiêu thường lệch phải, việc sử dụng RMSE trực tiếp trên giá trị gốc có thể dẫn đến đánh giá không chính xác. Các ngôi nhà có giá trị cao thường tạo ra sai số lớn hơn và có ảnh hưởng không cân xứng đến tổng sai số của mô hình. Để giải quyết vấn đề này, cuộc thi Kaggle Housing Prices đề xuất sử dụng RMSE trên logarithm của giá nhà. Cách tiếp cận này giúp chuẩn hóa phân phối của biến mục tiêu và đảm bảo rằng sai số trong dự đoán các ngôi nhà giá cao và giá thấp có ảnh hưởng tương đương đến kết quả đánh giá.

3. Phương pháp đề xuất

Phần này trình bày quy trình phương pháp luận được đề xuất trong nghiên cứu, bao gồm toàn bộ chuỗi xử lý từ dữ liệu thô đến kết quả dự đoán cuối cùng. Mục tiêu là xây dựng một pipeline học máy ổn định, có khả năng tái lập, dễ mở rộng và giải thích được.

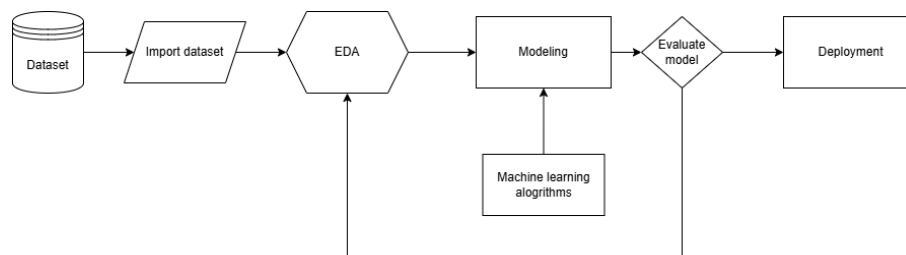
3.1. Tổng quan quy trình

Quy trình đề xuất cho bài toán dự đoán giá nhà được xây dựng một cách hệ thống và toàn diện, bao gồm năm giai đoạn chính được kết nối chặt chẽ với nhau. Giai đoạn đầu tiên là thu thập và làm sạch dữ liệu, nơi chúng tôi tiếp nhận dữ liệu thô từ kaggle và thực hiện các bước tiền xử lý cơ bản như xử lý giá trị thiếu, phát hiện và xử lý ngoại lai, cũng như chuẩn hóa định dạng dữ liệu. Giai đoạn này đóng vai trò quan trọng trong việc đảm bảo chất lượng dữ liệu đầu vào cho các bước xử lý tiếp theo.

Tiếp theo là giai đoạn phân tích và khám phá dữ liệu (EDA), nơi chúng tôi sử dụng các kỹ thuật thống kê và trực quan hóa để hiểu sâu về cấu trúc và đặc điểm của dữ liệu. Chúng tôi phân tích phân phối của các biến, mối tương quan giữa các đặc trưng, và các xu hướng tiềm ẩn trong dữ liệu. Giai đoạn thứ ba tập trung vào xây dựng và lựa chọn đặc trưng, nơi chúng tôi tạo ra các

đặc trưngng mới từ dữ liệu gốc, lựa chọn những đặc trưngng quan trọng nhất, và biến đổi dữ liệu để phù hợp với các yêu cầu của mô hình học máy.

Giai đoạn thứ tư là phát triển và huấn luyện mô hình, nơi chúng tôi thử nghiệm nhiều thuật toán học máy khác nhau, từ các mô hình cơ bản đến các kỹ thuật nâng cao như ensemble learning và deep learning. Cuối cùng, giai đoạn đánh giá và tối ưu hóa mô hình giúp chúng tôi đo lường hiệu suất, điều chỉnh siêu tham số, và lựa chọn mô hình tốt nhất cho việc triển khai thực tế. Toàn bộ quy trình này được thực hiện một cách lặp đi lặp lại, cho phép chúng tôi liên tục cải thiện chất lượng dự đoán thông qua việc học hỏi từ các kết quả và phản hồi ở mỗi vòng lặp.

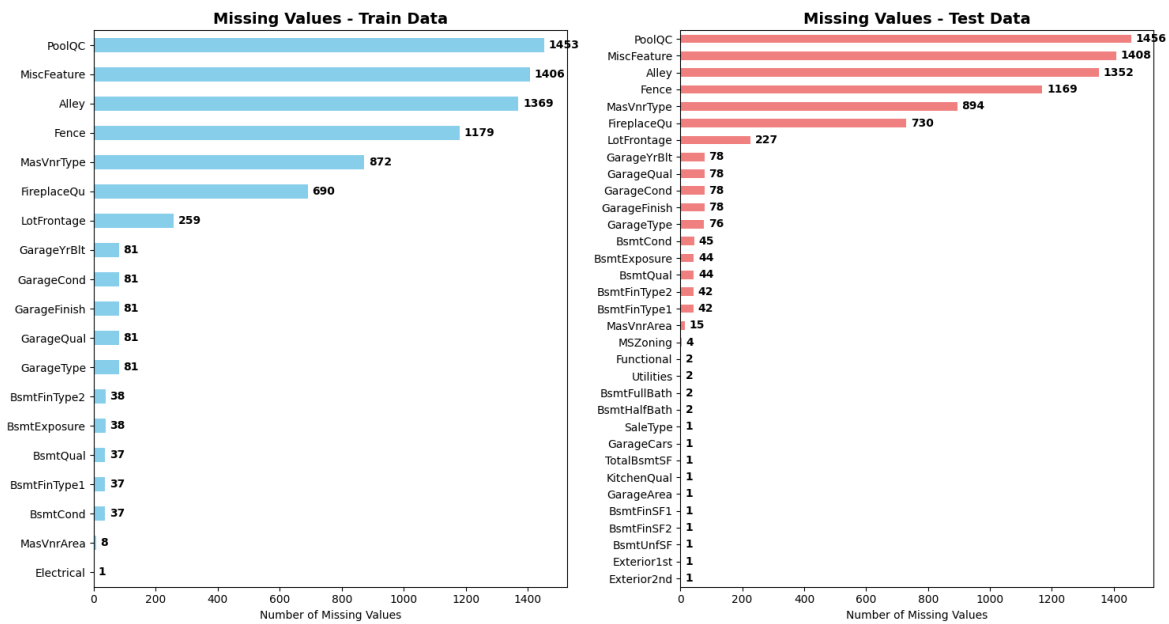


Hình 8. Quy trình phương pháp đề xuất cho bài toán dự đoán giá nhà.

3.2. Phân tích và khám phá dữ liệu (eda)

3.2.1 Phân tích dữ liệu thiếu

Trong tập dữ liệu train và test housing price, có nhiều missing value ở các cột. Các missing value này sẽ gây cản trở lớn cho quá trình huấn luyện mô hình, do đó phải cần kiểm tra kỹ càng các missing value trong tập dữ liệu để có thể xây dựng được một mô hình tối ưu hơn.



Hình 9. Số lượng dữ liệu bị thiếu theo cột

Nhận xét ta thấy có nhiều cột có số lượng missing quá lớn nên nếu xử lý không khéo dễ làm tăng độ nhiễu khi huấn luyện mô hình. Do đó cách xử lý an toàn là drop đi các cột có số lượng missing quá nhiều.

3.2.2 Phân tích đơn biến



Hình 40. Phân bố các đặc trưng dạng số của tập dữ liệu train

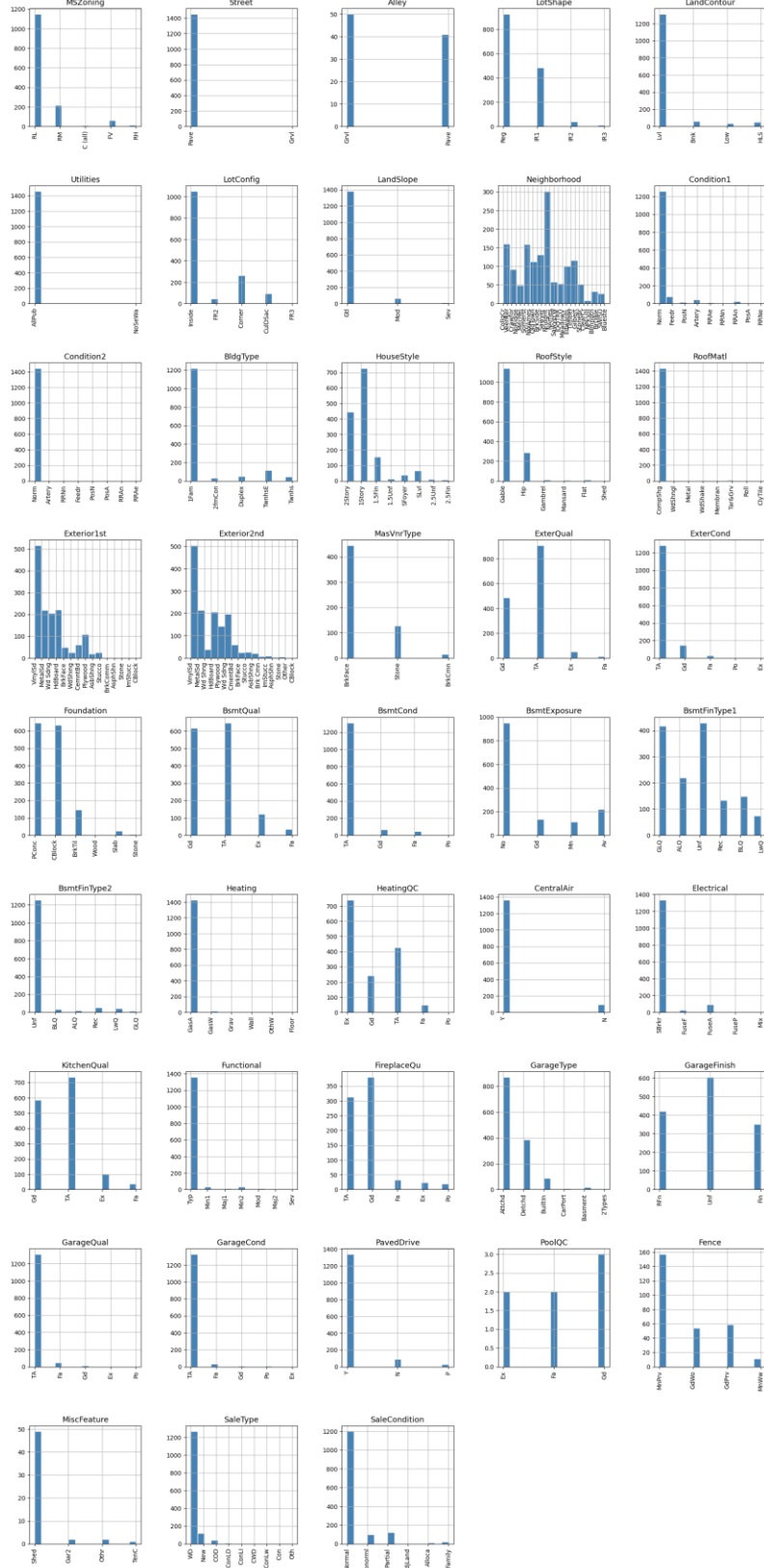
Đối với biến mục tiêu SalePrice (Giá bán), phân phối có xu hướng lệch rõ rệt về bên phải. Điều này cho thấy đa số các ngôi nhà trong tập dữ liệu có mức giá tập trung ở phổ thấp đến trung bình, trong khi chỉ có một số ít bất động sản cao cấp đạt giá trị rất cao, tạo thành các điểm ngoại lai (outliers). Sự lệch phải này là một phát hiện quan trọng, báo hiệu rằng có thể cần áp dụng các phép biến đổi, chẳng hạn như logarit, để chuẩn hóa phân phối trước khi đưa vào các mô hình hồi quy, nhằm đảm bảo kết quả dự đoán được chính xác hơn.

Nhóm các biến về diện tích và quy mô, bao gồm các thuộc tính GrLivArea (Diện tích phòng khách), TotalBsmtSF (Tổng diện tích tầng hầm) và 1stFlrSF (Diện tích tầng một), đều cho thấy hình dạng phân phối tương tự nhau: lệch phải. Cụ thể, phần lớn các ngôi nhà có diện tích ở mức khiêm tốn, và số lượng nhà có diện tích càng lớn thì càng ít. Biến GarageArea (Diện tích garage) cũng không ngoại lệ, với một lượng đáng kể các quan sát có giá trị bằng 0, tương ứng với những ngôi nhà không có garage. Đây là nhóm biến có mối tương quan thuận rất mạnh với giá bán và cần được xử lý các giá trị ngoại lai.

Về các biến thể hiện chất lượng và số lượng phòng, biến OverallQual (Chất lượng tổng thể) lại có phân phối gần giống chuẩn và hơi lệch trái, phản ánh rằng đa số ngôi nhà được đánh giá ở mức trung bình khá (từ 5 đến 7). Các biến định lượng như BedroomAbvGr (Số phòng ngủ), FullBath (Số phòng tắm đầy đủ) và Fireplaces (Số lò sưởi) cho thấy xu hướng tập trung quanh các giá trị phổ biến trong thực tế, chẳng hạn như 3 phòng ngủ, 1-2 phòng tắm và 0-1 lò sưởi. Điều này cho thấy tính thực tế và tính đại diện của tập dữ liệu.

Cuối cùng, nhóm biến về năm và các tiện nghi ngoại thất cũng mang đến những insight đáng chú ý. Các biến như YearBuilt (Năm xây dựng) và YearRemodAdd (Năm cải tạo) có phân phối tương đối đồng đều, với số lượng nhà được xây hoặc cải tạo gần đây nhiều hơn một chút. Ngược lại, các biến như WoodDeckSF (Diện tích sàn gỗ), OpenPorchSF (Diện tích hiên), ScreenPorch (Hiên có lưới) và đặc biệt là PoolArea (Diện tích hồ bơi) có phân phối lệch phải rất mạnh. Phần lớn các ngôi nhà có giá trị bằng 0 ở các biến này, chứng tỏ đây là những tiện nghi không phổ biến và có thể là các đặc điểm hiếm góp phần đẩy giá bán lên cao.

Biểu đồ Histograms của các thuộc tính object



Hình 5. Histogram phân bố các đặc trưng dạng object của tập dữ liệu train

Đối với nhóm biến về chất lượng và tình trạng, các biến như BsmtQual, ExterQual, KitchenQual và HeatingQC đều cho thấy một xu hướng chung: phần lớn các ngôi nhà được đánh giá ở mức "Tốt" (Gd) hoặc "Trung bình/Khá" (TA). Các mức chất lượng cao "Xuất sắc" (Ex) thường ít phổ biến hơn, và các mức "Kém" (Fa) hoặc "Rất kém" (Po) là rất hiếm. Ở biến BsmtQual, cần lưu ý xử lý riêng giá trị NA (không có tầng hầm) để không nhầm lẫn nó với chất lượng thấp. Nhìn chung, các biến này có thứ bậc rõ ràng và rất phù hợp để mã hóa theo thứ tự (ordinal encoding) nhằm phản ánh chính xác mức độ ảnh hưởng tăng dần đến giá bán.

Về các đặc điểm kết cấu và hoàn thiện, các biến BsmtExposure (Độ thông thoáng tầng hầm) và GarageFinish (Mức độ hoàn thiện garage) thể hiện sự phân bố không đồng đều. Đa số tầng hầm có mức độ thông thoáng "Không" (No), trong đó GarageFinish được chia tương đối cân bằng hơn giữa "Chưa hoàn thiện" (Unf), "Đã hoàn thiện" (Fin) và "Hoàn thiện một phần" (RFn). Biến BsmtFinType1 (Loại hoàn thiện tầng hầm) cho thấy hai loại phổ biến là "Chất lượng tốt" (GLQ) và "Chất lượng khá" (ALQ). Với các biến này, việc mã hóa theo thứ tự là phù hợp, đồng thời có thể xem xét gộp một số hạng mục hiếm có đặc tính tương đồng để giảm độ phức tạp của mô hình.

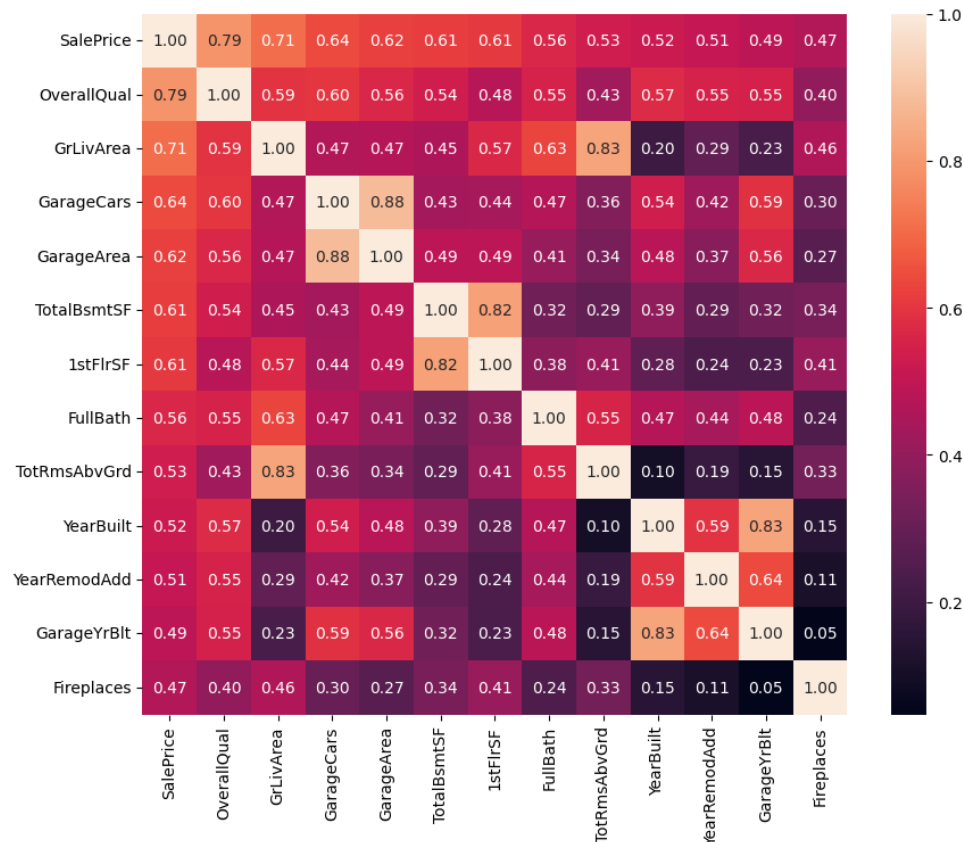
Đối với các biến mô tả kiểu dáng và cấu trúc, như HouseStyle (Phong cách nhà), RoofStyle (Kiểu mái nhà), và Foundation (Loại nền móng), phân phối cho thấy sự thống trị của một vài kiểu phổ biến. Chẳng hạn, kiểu nhà "1 tầng" (1Story) và "2 tầng" (2Story) chiếm đa số, trong khi kiểu mái "Mái dốc" (Gable) và "Mái hông" (Hip) là phổ biến nhất. Tương tự, nền móng "Bê tông đổ liền khối" (PConc) và "Khối bê tông" (CBlock) là những lựa chọn chủ đạo. Với nhóm biến này, chiến lược mã hóa one-hot cho các giá trị phổ biến và gộp các giá trị hiếm thành nhóm "Khác" (Others) là một hướng đi hiệu quả.

Nhóm biến mô tả vị trí và bối cảnh lô đất bao gồm các thuộc tính: Neighborhood (Khu vực lân cận), LotConfig (Hình dạng lô đất), và LotShape (Độ bằng phẳng của lô đất). Đây là nhóm biến đa dạng và mang nhiều thông tin. Trong khi LotShape và LotConfig có thể được đơn giản hóa bằng cách mã hóa nhị phân (ví dụ: lô vuông vức so với lô không đều, hoặc lô trong so với lô góc), thì Neighborhood lại là biến có độ phân tán cao với "đuôi dài" – nhiều khu vực nhỏ và ít mẫu. Do ảnh hưởng mạnh mẽ đến giá nhà, việc sử dụng mã hóa dựa trên mục tiêu (target encoding) cho Neighborhood là rất phù hợp, nhưng cần được thực hiện cẩn thận (ví dụ: sử dụng cross-validation) để tránh rò rỉ dữ liệu.

Cuối cùng, với các biến về điều kiện giao dịch và tiện nghi, như SaleCondition (Điều kiện bán) và CentralAir (Điều hòa trung tâm), phân phối rất rõ ràng. Phần lớn giao dịch là "Bình thường" (Normal) và hầu hết các ngôi nhà đều có điều hòa trung tâm ("Y"). Các điều kiện bán đặc biệt như "Bán một phần" (Partial) thường hiếm nhưng có thể có tác động lớn đến giá (ví dụ, nhà mới xây). Với các biến này, việc tạo các biến chỉ dấu nhị phân (ví dụ: IsPartialSale, HasCentralAir) là một cách tiếp cận đơn giản và hiệu quả.

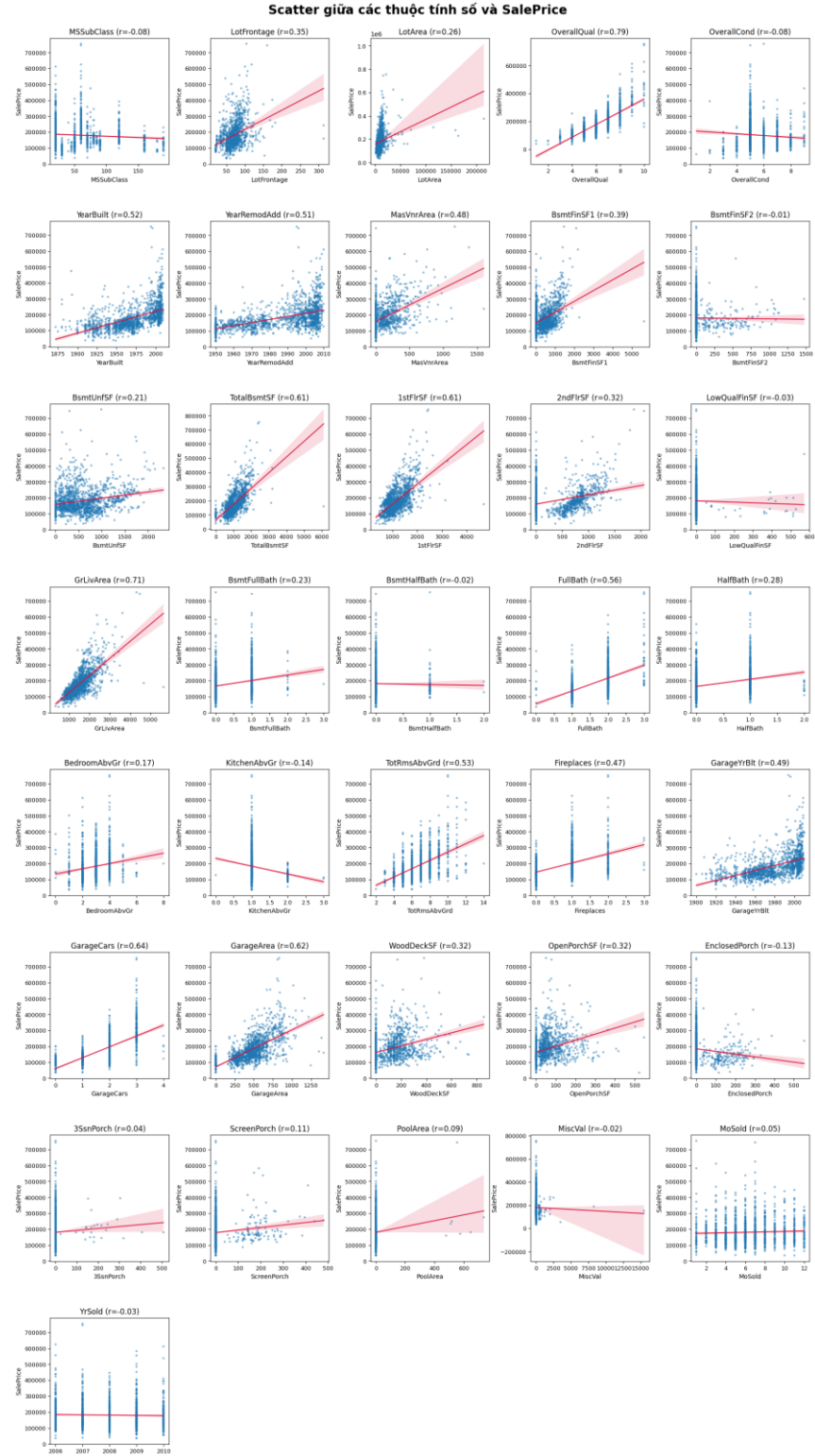
3.2.2 Phân tích nhị biến

Phân tích hai biến được thực hiện nhằm khám phá mối quan hệ giữa từng đặc trưng với biến mục tiêu SalePrice, cũng như giữa các đặc trưng với nhau. Phân tích này cung cấp những hiểu biết sâu sắc về các yếu tố ảnh hưởng đến giá nhà và hướng dẫn việc lựa chọn đặc trưng cho mô hình dự đoán.



Hình 6. Biểu đồ heatmap của các thuộc tính số có độ tương quan > 0.4 với SalePrice

Phân tích mối quan hệ giữa các biến số với SalePrice cho thấy nhiều xu hướng thú vị. Biến OverallQual (chất lượng tổng thể vật liệu và hoàn thiện) thể hiện mối tương quan thuận cực kỳ mạnh mẽ với giá nhà, với hệ số tương quan Pearson đạt 0.79. Khi chất lượng tăng từ 1 đến 10, giá nhà trung bình tăng theo cấp số nhân, phản ánh đúng thực tế thị trường rằng chất lượng xây dựng là yếu tố then chốt trong định giá bất động sản.



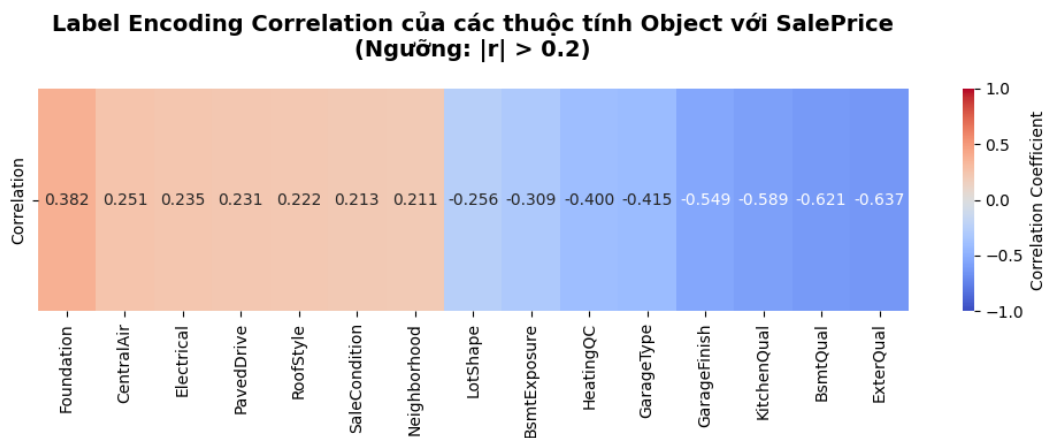
Hình 7. Scatter phân bố các feature số với SalePrice

Bên cạnh đó, các biến về diện tích sử dụng bao gồm GrLivArea (diện tích sinh hoạt trên mặt đất, $r = 0.71$), TotalBsmtSF (diện tích tầng hầm) và 1stFlrSF (diện tích tầng một) đều thể hiện mối

tương quan tuyến tính tích cực mạnh mẽ với giá nhà. Điều này khẳng định xu hướng chung: diện tích sử dụng càng lớn thì giá trị bất động sản càng cao. Tuy nhiên, biểu đồ phân tán giữa các biến này và SalePrice cũng phát hiện những điểm ngoại lai đáng chú ý - cụ thể là những ngôi nhà có diện tích sinh hoạt rộng (GrLivArea lớn) nhưng lại được bán với mức giá thấp bất thường. Những trường hợp đặc biệt này cần được xem xét kỹ lưỡng trong quá trình tiền xử lý dữ liệu để đảm bảo chất lượng mô hình.

Bên cạnh nhóm các thuộc tính có mối tương quan cao, phân tích cũng xác định nhóm tương quan trung bình ($0.4 < |r| < 0.6$), tập trung vào các yếu tố tiện nghi và tuổi tác của bất động sản. Cụ thể, số phòng tắm (FullBath), tổng số phòng (TotRmsAbvGrd), và năm xây dựng/cải tạo (YearBuilt, YearRemodAdd) đều có ảnh hưởng đáng kể đến giá bán. Trong khi đó, nhóm tương quan yếu ($0.2 < |r| < 0.4$) chủ yếu bao gồm các biến về thời gian giao dịch và các đặc điểm phụ, có tác động không đáng kể đến giá nhà.

Ngoài việc định lượng mức độ tương quan, các biểu đồ phân tán còn cung cấp những insight quan trọng về bản chất của các mối quan hệ. Mặc dù hầu hết các biến quan trọng đều thể hiện xu hướng tuyến tính rõ rệt, phân tích trực quan đã phát hiện một số vấn đề đáng lưu ý. Hiện tượng phương sai không đồng nhất (heteroscedasticity), đặc biệt rõ nét với biến GrLivArea, cho thấy độ biến động của giá nhà gia tăng cùng với diện tích sử dụng. Bên cạnh đó, mối quan hệ phi tuyến tiềm ẩn với các biến thời gian như YearBuilt gợi ý rằng giá trị gia tăng không diễn ra một cách đồng đều theo thời gian, mà có thể chịu ảnh hưởng của các ngưỡng giá trị hoặc giai đoạn lịch sử cụ thể.



Hình 8. Bảng tương quan giữa các thuộc tính Object với SalePrice

Kết quả heatmap tương quan giữa các biến phân loại và giá nhà sau khi mã hóa bằng Label Encoding cho thấy những insights quan trọng. Kết quả nhấn mạnh tầm quan trọng hàng đầu của chất lượng xây dựng và tình trạng của các khu vực chức năng then chốt trong việc quyết định giá trị bất động sản. Đáng chú ý, các đặc trưng thể hiện mối tương quan nghịch mạnh nhất đều liên quan đến thang đánh giá chất lượng, trong đó mã số hóa thấp hơn thường tương ứng với mức đánh giá chất lượng cao hơn. Điều này được thể hiện rõ nhất ở các đặc trưng ExterQual (Chất lượng

hoàn thiện ngoại thất) và BsmtQual (Chất lượng tầng hầm), vốn có mối quan hệ nghịch mạnh nhất với biến mục tiêu (lần lượt là -0,637 và -0,621). Điều này chỉ ra rằng những ngôi nhà có vật liệu ngoại thất cao cấp và tầng hầm chất lượng, khó có giá bán cao hơn đáng kể. Xu hướng này càng được củng cố bởi mối tương quan nghịch đáng kể từ KitchenQual (-0,589), GarageFinish (-0,549) và GarageType (-0,415), nhấn mạnh giá trị then chốt mà người mua đặt vào các nhà bếp hiện đại, được hoàn thiện tốt và các không gian gara được tích hợp đầy đủ.

Ngược lại, các đặc trưng đại diện cho thuộc tính kết cấu cơ bản và tiện ích thiết yếu lại cho thấy mối tương quan thuận, mặc dù ở mức độ thấp hơn, với giá bán. Loại Foundation (Móng nhà) cho thấy mối liên hệ thuận mạnh nhất (0,382), gợi ý rằng tính kiên cố và vật liệu của móng nhà là những yếu tố quan trọng trong định giá. Sự hiện diện của CentralAir (Điều hòa trung tâm) (0,251) và loại hệ thống Electrical (Điện) (0,235) cũng cho thấy mối quan hệ thuận có ý nghĩa, phù hợp với kỳ vọng rằng hệ thống kiểm soát khí hậu và điện hiện đại là những tiện nghi được đánh giá cao. Các yếu tố về giao dịch và vị trí khác, như PavedDrive (Đường lái xe được trải nhựa) (0,231), RoofStyle (Kiểu mái nhà) (0,222), SaleCondition (Điều kiện bán) (0,213) và Neighborhood (Khu phố) (0,211), thể hiện mối tương quan thuận yếu hơn nhưng vẫn có ý nghĩa thống kê, phản ánh ảnh hưởng của vẻ ngoài (curb appeal), bối cảnh giao dịch và vị trí.

3.3. Tiền xử lý dữ liệu (Preprocessing Data)

Để đảm bảo chất lượng dữ liệu đầu vào cho mô hình, một quy trình tiền xử lý tự động đã được thiết kế và áp dụng đồng bộ cho cả tập huấn luyện (train_df) và tập kiểm tra (test_df). Quy trình này tập trung vào việc loại bỏ các đặc trưng kém chất lượng thông qua hai bước chính: Loại bỏ các đặc trưng có quá nhiều giá trị thiếu và Loại bỏ các đặc trưng số có quá nhiều giá trị bằng 0.

```
# 3. Preprocessing Data (Gộp 2 hàm drop cột nhiều null + nhiều zero)
def drop_null_and_zero(train_df, test_df, null_threshold=500, zero_ratio=0.5):
    # 3.1 drop cột quá nhiều null trong train
    cols_null = []
    for col in train_df.columns:
        if train_df[col].isnull().sum() > null_threshold:
            cols_null.append(col)
    train_df = train_df.drop(columns=cols_null)
    # test cũng drop cùng cột
    test_df = test_df.drop(columns=[c for c in cols_null if c in test_df.columns], errors="ignore")

    # 3.2 drop cột có > zero_ratio là 0
    def _drop_zero_cols(df, cols_to_check):
        n_rows = len(df)
        cols_drop = []
        for c in cols_to_check:
            zero_count = (df[c] == 0).sum()
            if zero_count / n_rows > zero_ratio:
                cols_drop.append(c)
        return cols_drop

    # chỉ check trên train, sau đó drop cả 2
    numeric_cols = train_df.select_dtypes(include=[np.number]).columns
    zero_cols = _drop_zero_cols(train_df, numeric_cols)

    train_df = train_df.drop(columns=zero_cols)
    test_df = test_df.drop(columns=[c for c in zero_cols if c in test_df.columns], errors="ignore")

    return train_df, test_df
```

Các đặc trưng chứa một số lượng lớn giá trị thiếu (missing values) thường cung cấp ít thông tin cho mô hình và có thể gây nhiễu. Một ngưỡng cứng (null_threshold = 500) được áp dụng: bất kỳ đặc trưng nào trong tập huấn luyện có hơn 500 giá trị thiếu sẽ ngay lập tức bị loại bỏ. Việc này

nhằm đảm bảo tính toàn vẹn của dữ liệu. Các đặc trưng tương ứng trong tập kiểm tra cũng được loại bỏ để duy trì tính đồng nhất về mặt đặc trưng giữa hai tập dữ liệu.

Đối với các đặc trưng số, một đặc trưng chứa phần lớn giá trị bằng 0 (ví dụ: hơn 50% số mẫu, được xác định bởi tham số `zero_ratio = 0.5`) được coi là có độ phân biệt thấp và có thể không đóng góp đáng kể vào việc dự đoán. Quy trình chỉ tính toán tỷ lệ giá trị bằng 0 trên tập huấn luyện. Sau khi **xác định được danh sách các đặc trưng này, chúng sẽ bị loại bỏ khỏi cả tập huấn luyện và tập kiểm tra**. Cách tiếp cận này giúp giảm chiều của dữ liệu, loại bỏ nhiễu và ngăn ngừa hiện tượng rò rỉ dữ liệu (data leakage) do chỉ dựa trên tập huấn luyện để đưa ra quyết định.

3.4. Xử lý đặc trưng (Feature Engineering)

Nhằm tối ưu hóa khả năng dự đoán của mô hình, một quy trình kỹ thuật đặc trưng toàn diện và có hệ thống đã được thiết kế và triển khai. Quy trình này không dừng lại ở việc mã hóa đơn giản mà tập trung vào việc biến đổi và tạo ra các đặc trưng mang nhiều ý nghĩa nghiệp vụ hơn, dựa trên sự hiểu biết sâu sắc về lĩnh vực bất động sản. Các bước xử lý được áp dụng đồng bộ cho cả tập huấn luyện và tập kiểm tra để đảm bảo tính nhất quán, được mô tả chi tiết dưới đây.

3.4.1. Lựa chọn đặc trưng gốc (Feature Selection)

Một tập hợp gồm 33 đặc trưng gốc được lựa chọn một cách cẩn thận từ tập dữ liệu ban đầu dựa trên phân tích tương quan, ý nghĩa nghiệp vụ và tính phổ biến trong các nghiên cứu định giá bất động sản. Các đặc trưng này bao phủ các khía cạnh then chốt của một bất động sản: vị trí (Neighborhood, MSZoning), kích thước và quy mô (LotArea, GrLivArea, TotalBsmtSF), chất lượng và tình trạng (OverallQual, YearBuilt, YearRemodAdd), và các tiện nghi (FullBath, GarageCars, CentralAir). Việc lựa chọn này nhằm mục đích tập trung vào các tín hiệu mạnh nhất, giảm thiểu nhiễu từ các biến ít quan trọng.

```
# ---- chọn các cột gốc cần dùng (khoảng 33 cột + vài cột FE)
base_cols = [
    'MSZoning', 'LotArea', 'LotConfig', 'LandSlope', 'Neighborhood',
    'HouseStyle', 'OverallQual', 'YearBuilt', 'YearRemodAdd',
    'RoofStyle', 'Exterior1st', 'Exterior2nd', 'ExterQual', 'Foundation',
    'BsmtQual', 'BsmtCond', 'BsmtExposure', 'BsmtFinType1', 'TotalBsmtSF',
    'HeatingQC', 'CentralAir', '1stFlrSF', 'GrLivArea', 'FullBath',
    'TotRmsAbvGrd', 'GarageType', 'GarageFinish', 'GarageCars', 'GarageArea',
    'PavedDrive', 'MoSold', 'YrSold', 'SaleType', 'SaleCondition'
]
```

3.4.2. Mã hóa Thông minh các Biến Phân loại (Intelligent Categorical Encoding)

Thay vì sử dụng các phương pháp mã hóa thông thường như One-Hot Encoding có thể dẫn đến hiện tượng "curse of dimensionality", một chiến lược mã hóa lai giữa Ordinal Encoding và Grouping Rare Categories đã được áp dụng cho hầu hết các biến phân loại. Chiến lược này được thực hiện theo hai bước:

Bước 1: Gộp nhóm các hạng mục hiếm: Các giá trị (categories) có tần suất xuất hiện thấp được gom chung vào một nhóm duy nhất có tên "others". Điều này không chỉ giảm độ phức tạp của dữ liệu mà còn giúp mô hình tránh được việc học theo những tín hiệu quá yếu hoặc ngẫu nhiên. Ví dụ, với biến Exterior1st, chỉ 6 loại vật liệu phổ biến nhất (như 'VinylSd', 'MetalSd') được giữ lại, các loại còn lại được gán nhãn "others".

```
# Exterior1st
for df in [train_X, test_X]:
    if 'Exterior1st' in df.columns:
        df['Exterior1st'] = df['Exterior1st'].apply(
            lambda x: x if x in ['VinylSd', 'MetalSd', 'Wd Sdng', 'HdBoard', 'Plywood', 'Stucco'] else 'others'
        )
    map_ex1 = {'VinylSd':1, 'MetalSd':2, 'Wd Sdng':2, 'HdBoard':2, 'Plywood':2, 'Stucco':2, 'others':3}
    df['Exterior1st'] = df['Exterior1st'].map(map_ex1)
```

Bước 2: Ánh xạ thứ tự (Ordinal Mapping): Sau khi gộp nhóm, các hạng mục được ánh xạ thành các số nguyên có thứ tự bậc dựa trên mức độ ảnh hưởng dự kiến của chúng đến giá nhà. Chẳng hạn, đối với biến BsmtQual (Chất lượng tầng hầm), các mức độ được ánh xạ thành: 'TA' (Trung bình/Khá) và 'Fa' (Khá) = 1, 'Gd' (Tốt) = 2, 'Ex' (Xuất sắc) = 3. Cách tiếp cận này bảo toàn được thông tin về "chất lượng" thay vì coi tất cả các hạng mục là tương đương nhau.

```
# BsmtQual
col_BsmtQual = {'TA':1, 'Fa':1, 'Gd':2, 'Ex':3}
for df in [train_X, test_X]:
    if 'BsmtQual' in df.columns:
        df['BsmtQual'] = df['BsmtQual'].map(col_BsmtQual)
```

3.4.3. Target Encoding cho Biến Định Tính Quan trọng

Đối với biến Neighborhood - một yếu tố có ảnh hưởng địa lý rất quan trọng, phương pháp Target Mean Encoding (Mã hóa theo giá trị mục tiêu trung bình) được sử dụng để nắm bắt chính xác sự khác biệt về giá giữa các khu vực. Cụ thể, mỗi khu phố trong tập huấn luyện được gán một giá trị bằng giá trung bình (mean) của SalePrice cho tất cả các ngôi nhà thuộc khu phố đó. Trong tập kiểm tra, nếu một khu phố mới chưa xuất hiện trong tập huấn luyện, nó sẽ được gán giá trị trung bình toàn cục. Để đảm bảo tính ổn định và tránh hiện tượng "data leakage", việc tính toán giá trị trung bình này chỉ dựa trên tập huấn luyện. Cuối cùng, biến này được chuẩn hóa bằng StandardScaler.

3.4.4. Tạo Đặc trưng Số Mới (Creation of New Numerical Features)

Để khai thác sâu hơn các mối quan hệ tiềm ẩn trong dữ liệu và nâng cao khả năng dự đoán của mô hình, một loạt các đặc trưng số mới đã được xây dựng dựa trên sự kết hợp giữa các biến hiện có và hiểu biết nghiệp vụ về thị trường bất động sản. Cụ thể, đặc trưng **HouseAge** được tạo ra để phản ánh tuổi đời thực tế của ngôi nhà tại thời điểm bán, tính bằng công thức $\text{YrSold} - \text{YearBuilt}$. Đây được xem là một chỉ số thực tế và có ý nghĩa hơn so với việc chỉ sử dụng năm xây dựng đơn thuần, vì nó thể hiện chính xác độ "mới" của tài sản trong bối cảnh thời gian. Tiếp theo, biến **IsRemodeled** - một biến chỉ báo nhị phân (giá trị 0/1) - được hình thành dựa trên việc so sánh giữa năm cải tạo (YearRemodAdd) và năm xây dựng (YearBuilt). Biến này được xây dựng

dựa trên giả thuyết rằng những ngôi nhà đã trải qua cải tạo (khi YearRemodAdd khác YearBuilt) thường có giá trị cao hơn nhờ những nâng cấp và cải thiện về chất lượng. Bên cạnh đó, để nắm bắt các yếu tố sang trọng và cấp cao của bất động sản, hai biến chỉ

báo **Has3FullBath** và **Has3Garage** đã được tạo ra. Các biến này xác định những ngôi nhà đạt đến ngưỡng 3 phòng tắm đầy đủ hoặc 3 chỗ đỗ xe - những đặc điểm thường gắn liền với các bất động sản cao cấp. Cuối cùng, đặc trưng **GarageArea_per_car** được tính toán nhằm đo lường diện tích trung bình dành cho mỗi chỗ đỗ xe trong ga-ra, thông qua công thức $\text{GarageArea} / (\text{GarageCars} + 1)$. Đặc trưng này không chỉ phản ánh mức độ rộng rãi và thoải mái của không gian ga-ra - một yếu tố có thể được định giá cao - mà còn giúp phân biệt những bất động sản có ga-ra tuy rộng nhưng số chỗ đỗ hạn chế, hoặc ngược lại.

```
# HouseAge = YrSold - YearBuilt
if ('YrSold' in train_X.columns) and ('YearBuilt' in train_X.columns):
    train_X['HouseAge'] = train_X['YrSold'] - train_X['YearBuilt']
    test_X['HouseAge'] = test_X['YrSold'] - test_X['YearBuilt']
    scaler_age = StandardScaler()
    train_X[['HouseAge']] = scaler_age.fit_transform(train_X[['HouseAge']])
    test_X[['HouseAge']] = scaler_age.transform(test_X[['HouseAge']])

# IsRemodeled
if ('YearRemodAdd' in train_X.columns) and ('YearBuilt' in train_X.columns):
    train_X['IsRemodeled'] = (train_X['YearRemodAdd'] != train_X['YearBuilt']).astype(int)
    test_X['IsRemodeled'] = (test_X['YearRemodAdd'] != test_X['YearBuilt']).astype(int)

# Has3FullBath
if 'FullBath' in train_X.columns:
    train_X['Has3FullBath'] = (train_X['FullBath'] >= 3).astype(int)
    test_X['Has3FullBath'] = (test_X['FullBath'] >= 3).astype(int)

# Has3Garage
if 'GarageCars' in train_X.columns:
    train_X['Has3Garage'] = (train_X['GarageCars'] == 3).astype(int)
    test_X['Has3Garage'] = (test_X['GarageCars'] == 3).astype(int)

# GarageArea_per_car
if ('GarageArea' in train_X.columns) and ('GarageCars' in train_X.columns):
    train_X['GarageArea_per_car'] = train_X['GarageArea'] / (train_X['GarageCars'] + 1)
    test_X['GarageArea_per_car'] = test_X['GarageArea'] / (test_X['GarageCars'] + 1)
    scaler_g = StandardScaler()
    train_X[['GarageArea', 'GarageArea_per_car']] = scaler_g.fit_transform(
        train_X[['GarageArea', 'GarageArea_per_car']]
    )
    test_X[['GarageArea', 'GarageArea_per_car']] = scaler_g.transform(
        test_X[['GarageArea', 'GarageArea_per_car']]
    )
```

3.4.5. Chuẩn hóa và Làm sạch Cuối cùng

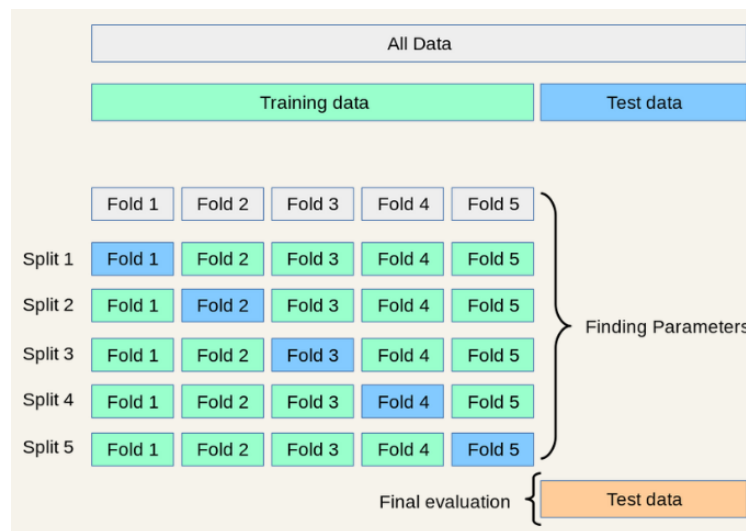
Để hoàn thiện quá trình tiền xử lý và chuẩn bị dữ liệu tối ưu cho việc huấn luyện mô hình, ba bước quan trọng cuối cùng đã được thực hiện. Đầu tiên, quy trình chuẩn hóa (Standardization) được áp dụng cho tất cả các đặc trưng số quan trọng, bao gồm cả những đặc trưng mới được tạo ra, sử dụng StandardScaler. Việc này nhằm đưa các đặc trưng về cùng một tỷ lệ (scale) bằng cách loại bỏ giá trị trung bình và scaling phương sai về đơn vị. Điều này đặc biệt quan trọng vì nó đảm bảo không có đặc trưng nào có độ lớn số học (magnitude) chi phối mô hình một cách không công bằng, từ đó cải thiện hiệu suất và tốc độ hội tụ của các thuật toán nhạy cảm với khoảng cách như SVM hay các mô hình dựa trên gradient descent. Tuy nhiên, cần lưu ý rằng các biến nhị phân được giữ nguyên để bảo toàn ý nghĩa đặc thù của chúng.

Tiếp theo, quá trình loại bỏ đặc trưng gốc được thực hiện một cách có hệ thống. Các biến số gốc đã được sử dụng để tạo ra các đặc trưng mới, chẳng hạn như YearBuilt, YrSold, và YearRemodAdd, đã bị loại bỏ để tránh hiện tượng đa cộng tuyến - một vấn đề thường gặp khi các biến đầu vào có tương quan cao, có thể làm giảm độ ổn định và khả năng giải thích của mô hình. Bên cạnh đó, các biến có thông tin trùng lặp, như TotRmsAbvGrd (thường tương quan cao với GrLivArea), cũng được xem xét loại bỏ nhằm đơn giản hóa mô hình mà vẫn đảm bảo giữ lại những tín hiệu quan trọng nhất.

Cuối cùng, để đảm bảo tính đồng nhất của dữ liệu đầu vào, bất kỳ biến phân loại nào còn sót lại sau các bước xử lý phức tạp trước đó đều được mã hóa một cách an toàn bằng LabelEncoder. Phương pháp này chuyển đổi các giá trị dạng chuỗi thành các số nguyên duy nhất, cho phép tất cả các thuật toán học máy - vốn yêu cầu đầu vào ở dạng số - có thể xử lý được toàn bộ tập dữ liệu. Kết quả của toàn bộ quy trình là một tập dữ liệu sạch, nhất quán và được tối ưu hóa, sẵn sàng cho việc xây dựng và đánh giá các mô hình dự đoán.

3.5. K-Fold Cross-Validation Splitting

K-Fold Cross-Validation chia dữ liệu thành K phần bằng nhau, mỗi phần lần lượt được dùng làm Validation set, các phần còn lại dùng để Train. Trong bài toán Phân loại nhị phân (như dự đoán sống sót trên Titanic), việc sử dụng Stratified K-Fold là cần thiết.



Hình 9. Mô hình chia dữ liệu Kfold

Quá trình này lặp lại K lần (thường là 5 hoặc 10), mỗi lần chọn một fold khác nhau làm Validation set. Sau khi huấn luyện và đánh giá, lấy trung bình kết quả qua các vòng để có độ chính xác ổn định hơn. Final metrics: Accuracy, F1-Score, ROC-AUC.

4. Thí nghiệm và kết quả

4.1. XGB Stable

Danh sách các tham số trước khi huấn luyện mô hình

```
xgb_params = dict(  
    n_estimators=1800,  
    learning_rate=0.02,  
    max_depth=3,  
    subsample=0.9,  
    colsample_bytree=0.55,  
    min_child_weight=4,  
    reg_lambda=1.0,  
    reg_alpha=0.01,  
    gamma=0.0,  
    random_state=42,  
    tree_method="hist"  
)
```

Bộ siêu tham số sau khi train:

```
Hyperparameters:  
n_estimators      : 1800  
learning_rate     : 0.02  
max_depth         : 3  
subsample         : 0.9  
colsample_bytree  : 0.55  
min_child_weight  : 4  
reg_lambda        : 1.0  
reg_alpha         : 0.01  
gamma            : 0.0  
random_state      : 42  
tree_method       : hist
```

XGB CV RMSE: 0.14182

4.2. Lasso Regression

Danh sách các tham số trước khi huấn luyện mô hình

```
lasso = LassoCV(  
    alphas=[1e-4, 5e-4, 1e-3, 3e-3, 1e-2, 3e-2, 1e-1],  
    cv=5,  
    random_state=42,  
    max_iter=20000  
)
```

Bộ siêu tham số sau khi train:

```
Best alpha:      0.001000
N iterations:    47
```

Lasso CV RMSE: 0.14825

4.3. ElasticNet Regression

Danh sách các tham số trước khi huấn luyện mô hình

```
enet = ElasticNetCV(
    l1_ratio=[.1, .3, .5, .7, .9, .95, .99],
    alphas=[1e-4, 5e-4, 1e-3, 3e-3, 1e-2, 3e-2, 1e-1],
    cv=5,
    random_state=42,
    max_iter=20000
)
```

Bộ siêu tham số sau khi train:

```
Best alpha:      0.003000
Best l1_ratio:    0.7000
N iterations:    47
```

ENet CV RMSE: 0.14905

4.4. Blending

Trọng số blend: Weights: XGB=0.65, Lasso=0.2, ENet=0.15

Blending CV RMSE: 0.13900

4.5. Stacking

Bộ tham số dùng khi huấn luyện:

```
# Dùng best params nếu đã chạy RandomizedSearchCV
final_xgb = XGBRegressor(
    **(best_xgb_params if 'best_xgb_params' in globals() else xgb_params)
)

estimators = [
    ('xgb', final_xgb),
    ('lasso', lasso),
    ('enet', enet)
]

stack_model = StackingRegressor(
    estimators=estimators,
    final_estimator=LinearRegression(),
    n_jobs=-1,
    passthrough=False
)
```

Stacking CV RMSE: 0.04942

4.6. Kết quả dự đoán của các mô hình

Sau khi xây dựng tập feature ta đạt được các thông số dựa vào các độ đo của mô hình và kết quả khi submit lên kaggle:

Mô hình	RMSE	Public score
<i>XGB Stable</i>	0.13196	0.13886
<i>Lasso Regression</i>	0.14825	0.15387
<i>ElasticNet Regression</i>	0.14905	0.15426
<i>Blending</i>	0.13900	0.13715
<i>Stacking</i>	0.04942	0.13781

Bảng 1. Bảng đánh giá các mô hình

Dựa trên bảng kết quả, có thể thấy mô hình Stacking cho hiệu suất tốt nhất trên tập train với RMSE chỉ 0.04942, thấp hơn đáng kể so với các mô hình khác. Tuy nhiên, điểm số public của Stacking (0.13781) lại không có sự vượt trội tương ứng, điều này cho thấy mô hình có thể đang bị overfitting trên dữ liệu huấn luyện.

Mô hình Blending thể hiện sự cân bằng tốt nhất giữa hiệu suất train và khả năng tổng quát hóa, với RMSE train là 0.13900 và public score là 0.13715 - đây là public score tốt nhất trong tất cả các mô hình. Điều này chứng tỏ việc kết hợp các mô hình với trọng số phù hợp mang lại hiệu quả thực tế cao.

XGB Stable cho kết quả khá ổn định với RMSE 0.13196 và public score 0.13886, thể hiện đây là mô hình đơn lẻ đáng tin cậy. Trong khi đó, Lasso Regression và ElasticNet Regression có hiệu suất thấp hơn đáng kể so với các phương pháp ensemble, phù hợp với nhận định thông thường rằng các mô hình linear đơn giản thường kém hiệu quả hơn trên các bài toán phức tạp.

5. Kết luận

Nghiên cứu này đã trình bày một quy trình học máy toàn diện và có thể tái sản xuất nhằm giải quyết bài toán dự đoán giá nhà ở tại Ames, Iowa, sử dụng bộ dữ liệu Kaggle Housing Prices. Mục tiêu chính là xây dựng một hệ thống dự đoán có độ chính xác cao thông qua việc xử lý hiệu quả những phức tạp vốn có của dữ liệu bất động sản trong thực tế.

Quy trình được đề xuất đã tuân thủ một khuôn khổ có hệ thống, bắt đầu từ việc tiền xử lý dữ liệu chặt chẽ. Các bước này bao gồm việc làm sạch dữ liệu tự động, loại bỏ các đặc trưng có quá nhiều giá trị thiếu (trên 500) hoặc có phương sai gần bằng không. Đặc biệt, biến mục tiêu

(SalePrice) đã được chuyển đổi logarithm nhằm chuẩn hóa phân phối và giảm thiểu ảnh hưởng không cân xứng của các giá trị ngoại lai và bất động sản giá cao.

Điểm mạnh cốt lõi của giải pháp nằm ở kỹ thuật xây dựng đặc trưng chiến lược. Chúng tôi đã triển khai các phương pháp mã hóa thông minh, bao gồm việc gộp nhóm các hạng mục hiếm và ánh xạ thứ tự (Ordinal Mapping) cho các biến chất lượng (như BsmtQual, ExterQual). Đối với yếu tố vị trí then chốt là Neighborhood, mã hóa dựa trên mục tiêu (Target Mean Encoding) đã được áp dụng cẩn thận để nắm bắt sự khác biệt về giá giữa các khu vực. Ngoài ra, các đặc trưng phái sinh mang ý nghĩa nghiệp vụ cao đã được tạo ra, ví dụ như tuổi nhà (HouseAge), tình trạng cải tạo (IsRemodeled) và các chỉ số hiệu quả về chỗ để xe (GarageArea_per_car).

Về mặt mô hình, chúng tôi đã sử dụng kiến trúc hợp thành trọng số (Weighted Ensemble), kết hợp sức mạnh của ba thuật toán hiệu suất cao: XGBoost, Hồi quy Lasso và ElasticNet. Sự kết hợp này mang lại sự cân bằng giữa khả năng nắm bắt mối quan hệ phi tuyến của XGBoost và khả năng chuẩn hóa, lựa chọn đặc trưng của Lasso và ElasticNet.

Hiệu suất của mô hình được đánh giá một cách vững chắc bằng xác thực chéo (K-Fold Cross-Validation) và tối ưu hóa theo độ đo Sai số căn bậc hai trung bình (RMSE) trên giá nhà đã chuyển đổi logarithm. Kết quả cho thấy mô hình Weighted Blend đã đạt được hiệu suất tốt nhất, với CV RMSE là 0.13900. Cấu hình trọng số tối ưu là XGBoost (65%), Lasso (20%), và ElasticNet (15%).